

VProfile Multi-Tier Application

Project Documentation & Operational Guide | Local, Kubernetes & CI/CD Setup

The **VProfile Web Application** is a robust multi-tier stack built using Java Spring, MySQL, Memcached, RabbitMQ, Tomcat, and NGINX. This repository provides complete configuration specs for containerized local development with Docker Compose, production orchestration on Kubernetes, and automated deployment via Jenkins.

1. Stack & Component Architecture

The application architecture decouples static web serving, dynamic application processing, caching, message queuing, and persistence into distinct services:

Service Name	Technology	Default Port	Role & Description
vproweb	NGINX	80	Web server & Reverse Proxy routing incoming web requests.
vproapp	Spring MVC / Tomcat	8080	Java Web Application processing core business logic.
vprodb	MySQL 8.0	3306	Relational Database storing application datasets.
vprocache01	Memcached	11211	High-performance memory object caching system.
vpromq01	RabbitMQ	15672 / 5672	Message broker for asynchronous task management.

2. Repository Directory Structure

```
.
├── Docker-files/
│   ├── app/           # Tomcat application Dockerfile
│   ├── db/            # MySQL custom database initialization Dockerfile
│   └── web/            # NGINX reverse proxy configuration
├── docker-compose.yml  # Multi-container local orchestration
├── Jenkinsfile          # Continuous Integration & Deployment pipeline
├── pom.xml              # Maven build configuration and dependencies
├── k8s/                 # Kubernetes manifests
│   ├── app-secret.yml   # Base64 encoded secrets (DB/RMQ credentials)
│   ├── db-CIP.yml       # MySQL ClusterIP service
│   ├── mc-CIP.yml       # Memcached ClusterIP service
│   ├── mcdep.yml        # Memcached Deployment
│   ├── rmq-CIP-service.yml # RabbitMQ ClusterIP service
│   ├── rmq-dep.yml      # RabbitMQ Deployment
│   ├── vproapp-service.yml # App Service configuration
│   ├── vproappdep.yml   # App Deployment
│   └── vprodbdep.yml    # MySQL Database Deployment
```

3. Prerequisites & Dependencies

- **Runtime:** Java 8 JDK & Apache Maven 3.x
- **Containerization:** Docker Engine v20+ & Docker Compose v2+

- **Orchestration:** Kubernetes Cluster (Minikube / EKS / GKE) & `kubectl` CLI
- **CI/CD Server:** Jenkins with Docker and Kubernetes plugins configured

4. Local Development (Docker Compose)

To run the entire multi-tier stack locally inside isolated containers:

1. Clone the repository:

```
git clone https://github.com/Gamal-Mohammad/iti_cicd.git
cd iti_cicd
```

2. Spin up containers in detached mode:

```
docker-compose up -d --build
```

3. Verify running endpoints:

- **Web Application (NGINX):** `http://localhost:80`
- **Direct App Engine (Tomcat):** `http://localhost:8080`
- **RabbitMQ Dashboard:** `http://localhost:15672` (User: `guest` | Pass: `guest`)

5. Local Build & Test (Maven)

If you wish to test or package the application artifact manually without Docker:

```
# Package application into a .war file
mvn clean package

# Run unit tests and generate JaCoCo code coverage report
mvn test
```

6. Deployment to Kubernetes

Apply the Kubernetes manifests in sequence to provision the stack:

```
kubectl apply -f app-secret.yml
kubectl apply -f db-CIP.yml
kubectl apply -f mc-CIP.yml
kubectl apply -f mcdep.yml
kubectl apply -f rmq-CIP-service.yml
kubectl apply -f rmq-dep.yml
kubectl apply -f vproapp-service.yml
kubectl apply -f vproappdep.yml
kubectl apply -f vprodbdep.yml
```

7. CI/CD Pipeline (Jenkins)

The included `Jenkinsfile` defines a multi-stage automation pipeline:

1. **Checkout:** Pulls code from the `main` branch.
2. **Build App & DB Images:** Builds container images using Dockerfiles under `Docker-files/`.

3. **Push to Docker Hub:** Authenticates via Jenkins credentials (`dockerhub`) and pushes tagged images.
4. **Deploy to Kubernetes:** Executes `kubect1 apply` commands to roll out changes to the Kubernetes cluster.